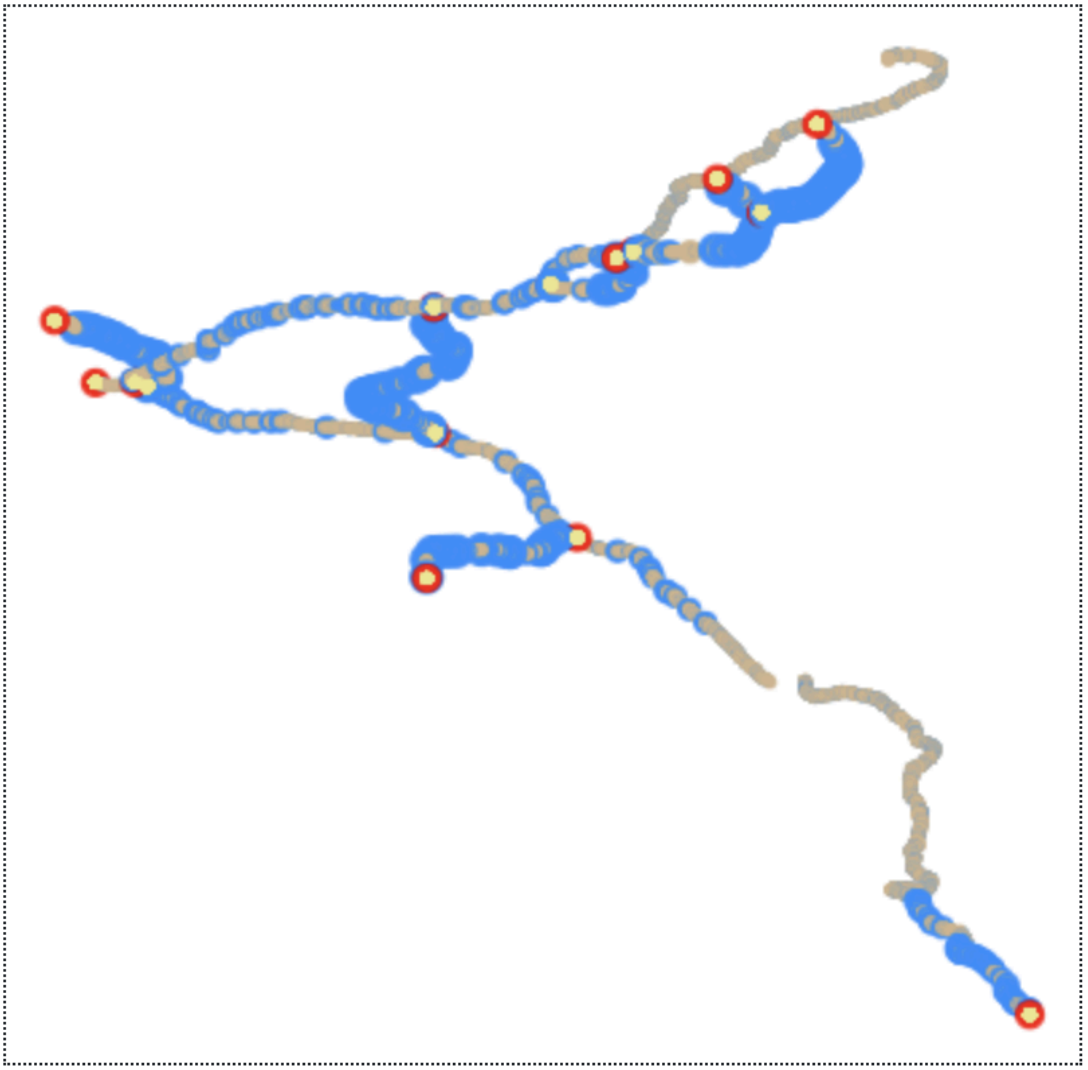
Assignment 3. Network Model Generation



EPA133a. Advanced Simulation

Dr. Yilin Huang, Prof. dr.ir. Alexander Verbraeck

TU Delft, Multi Actor Systems Department

By Group 11

Jonathan Vermeulen - 5144434
Stijn Keukens - 5072700
Scipio Bruijn - 5868181
Evi de Kok - 5878179
Annette Dorresteijn - 5868629

Introduction

This report presents the development of a simulation model for goods transport over the main road network in Bangladesh, focusing on National Highways N1 and N2 and their connected side roads longer than 25 km. Using Mesa 2.1.4 and NetworkX, the model was automatically generated from road and bridge data, with intersections placed at appropriate locations to correctly connect the roads. Vehicles are generated every 5 minutes from each “sourcesink” in the network and routed to a random destination using NetworkX shortest-path routing, with results cached to improve performance. The model was run for 5 × 24 hours across five scenarios in which bridges in different condition categories (A–D) break down at increasing probabilities. Travel time and delays were recorded for each scenario to analyse the impact of bridge failures on network performance. Based on these results, the following analysis might identify critical bridges to invest in to maximize travel productivity.

Contents

Introduction **1**
Contents **2**
1. Data preparation **2**
2. Model design & Adaptation **2**
3. Results **2**
4. Reflection **2**
5. Acknowledgement of AI & Group Work **2**
5.1 AI Statement 2
5.2 Group Work Contribution 3
References **3**

1. Data preparation

The network was constructed from two raw datasets: `BMMS_overview.xlsx`, containing all bridges in Bangladesh with their location, length, and structural condition (rated A to D), and `_roads3.csv`, containing road points with chainage values (distance in km from the road origin) and geographical coordinates (latitude/longitude) for each road.

To define the scope of the network, all roads with names of the form N1XX and N2XX were first considered. For each of these roads, the total length was determined, and only those exceeding 25 km were included in the network. For each selected road, the raw road points were converted into discrete link segments representing the stretch of road between two consecutive points. Bridge locations were extracted from the `BMMS_overview` dataset and inserted into the road network according to their chainage values. To prevent spatial overlap, any link segment overlapping with a bridge was split at the bridge boundaries, so that bridges and road links were represented as distinct, non-overlapping objects.

After this selection, intersections between the side roads and the main roads, N1 and N2, were identified using geographic coordinates, where two roads were within 0.02 degrees of each other. In cases where a side road did not contain a direct coordinate-based intersection with N1 or N2, the endpoint of that side road closest to N1 or N2 was taken as the connecting point and inserted into the corresponding location on the main road, ensuring all selected side roads were connected to the main network. At each such junction, an intersection node was added to facilitate routing between roads, along with a SourceSink node to generate and absorb vehicles. SourceSink nodes were also placed at open road endpoints that were not connected to any other road. SourceSinks are included at every side road end, including intersections, to reflect reality, where cars can come from many different starting points.

Finally, duplicate nodes were removed, all length values were converted from kilometres to metres, and each network object was assigned a unique identifier starting from 1,000,000. The resulting network was exported as network_AS3.csv, which serves as the primary input for the simulation model.

2. Model design & Adaptation

2.1 Model Design

2.1.1 Design

The modelling workflow consists of several interconnected scripts responsible for network construction, simulation logic, and experiment execution. The network itself is generated from the raw infrastructure datasets using a dedicated preprocessing script that integrates road geometry from `_roads3.csv` with bridge information from `BMMS_overview.xlsx`. This preprocessing step constructs a simulation-ready infrastructure network consisting of links, bridges, intersections, and source–sink nodes, which is exported as `network_AS3.csv`.

The simulation model then uses this generated network as input. The infrastructure components and vehicle behaviour are implemented in `components.py`, which defines the different agent classes and their interactions within the network. The main model logic is implemented in `model.py`, which reads the network dataset, initializes the infrastructure agents, manages the simulation scheduler, and records key performance indicators such as travel times and delays.

Experimental execution is controlled by `model_run.py`, which defines simulation scenarios, sets reproducibility seeds, runs multiple replications, and exports the resulting data for further analysis. In addition, `model_viz.py` provides a graphical interface that allows the model to be explored interactively, enabling visual inspection of vehicle movements and network dynamics.

This modular design separates data preparation, model logic, experimentation, and visualization, improving transparency and reproducibility while allowing individual components of the model to be modified without affecting the overall structure.

2.1.2 Model Assumptions

A key assumption in this model concerns the placement of SourceSink nodes at intersection locations. In reality, vehicles entering the network from side roads would do so at the actual entry point of that side road, which may not coincide exactly with the intersection of two national highways. For simplicity, it was assumed that vehicles can enter and exit the network at every intersection, and SourceSink nodes were therefore placed at the same coordinates as intersection nodes. This means that in the model, every junction between two roads serves as both a routing point and a potential origin or destination for vehicles. This is done because the current selection is a simplified road network with limited entry points; by including more entry points, it simulates a more dynamic network.

This assumption has an important implication for how vehicles behave in the simulation. Because SourceSink nodes are present not only at road endpoints but also at intermediate intersections, a vehicle could theoretically be removed from the network the moment it passes through any intersection along its route. To prevent this, the model was adjusted so that a vehicle is only removed

when it reaches the SourceSink of its intended destination, not merely any SourceSink it encounters along the way. Without this adjustment, vehicles would frequently exit the network at intermediate intersections rather than completing their full trip, which would lead to artificially short travel times and an inaccurate representation of network performance.

2.2 Model Adaptation

2.2.1 Model_run.py

In the original implementation, the script ran a single simulation with a fixed random seed and printed the results to the terminal. The revised version introduces a multi-scenario, multi-run experiment framework, executing 5 scenarios (0–4) with 10 replications each, for a total of 50 simulation runs. For each run, the random seed is varied using a base seed combined with the run index to ensure statistical variation while maintaining reproducibility.

To ensure independence between runs, the global vehicle counter used for truck identifiers is reset before each simulation. The script also records structured travel metrics, discussed in section 2.3.

Simulation outputs are automatically written to CSV files in a dedicated output directory. Results are stored per scenario, combined across all runs, and summarized using descriptive statistics such as mean, median, standard deviation, and total travel time. Key experiment parameters, including the number of scenarios, runs, simulation duration, and base seed, are defined as configurable constants at the top of the script.

2.2.2 Model_viz.py

The `model_viz.py` script provides an interactive visualization of the simulation model using the Mesa server interface. It renders the infrastructure network and vehicle agents on a canvas, allowing the movement of vehicles and the state of infrastructure components to be observed during the simulation.

The visualization was extended to support the newly introduced SourceSink nodes, which represent locations that can both generate and absorb vehicles. A layering system was also implemented to control drawing order, ensuring that important nodes such as sources, sinks, and SourceSink agents are rendered above infrastructure elements like links.

The portrayal rules for infrastructure were further refined by introducing separate visualization logic for bridges and links. Agent size now scales with the number of vehicles present while remaining capped to prevent oversized markers. In addition, bridge colors change dynamically depending on whether vehicles are present, making congestion visually identifiable. The canvas size was increased from 400×500 to 500×500 to better accommodate the expanded road network. Finally, a scenario parameter was added to the server configuration to allow the visualization to align with the multi-scenario experiment setup, while text labels for infrastructure agents were suppressed to reduce visual clutter.

2.2.3 Model.py

The `model.py` file implements the core simulation logic and was extended to support a more flexible and realistic network-based traffic model. A key addition is the introduction of a NetworkX directed graph that represents the road network. Nodes and edges are constructed from the infrastructure dataset, with edge weights based on segment length. This allows vehicles to compute shortest paths across the entire network rather than following predefined routes, enabling realistic routing between different roads.

To support multi-road connectivity, the model automatically detects junctions between roads by identifying intersections and SourceSink nodes that are geographically close. These nodes are connected with zero-weight edges in the graph, allowing vehicles to move between roads during route computation. The model initialization was also extended with a scenario parameter, enabling different infrastructure conditions to be simulated, and with a travel time collector that records completed vehicle trips for later analysis.

Additional improvements were made to increase robustness and flexibility. Road names are now loaded dynamically from the network dataset, removing the need for hardcoded road lists. File paths are constructed using the `pathlib` module to ensure that the network file can be located reliably regardless of the execution location. Route computation was further optimized by introducing path caching, which stores previously computed shortest paths and reuses them when the same origin–destination pair occurs again. Finally, diagnostic information about the generated graph, such as the number of nodes, edges, and connected components, is printed during initialization to assist with verifying network connectivity.

2.2.4 Components.py

The `components.py` file defines the infrastructure and vehicle agent classes and was extended to support more realistic traffic dynamics and experimental analysis. A key addition is a scenario-based bridge breakdown system, implemented through a dictionary of breakdown probabilities for different bridge condition grades (A–D). Five scenarios are defined, each assigning increasing probabilities that a bridge is permanently broken at initialization. This mechanism allows infrastructure reliability to vary across scenarios, forming the basis of the experimental analysis.

Bridge delays were also redesigned to produce length-dependent delay times. If a bridge is broken, the delay experienced by vehicles is now drawn from a probability distribution scaled to the physical length of the bridge, making delays proportional to bridge size rather than arbitrary.

To support experimental output, the Sink component was modified to record completed trips before removing vehicles from the simulation. Each record includes the vehicle identifier, generation step, removal step, travel time, and originating source node. Several robustness improvements were also introduced to ensure stable simulation runs. These include validating vehicle routes during generation, adding bounds and null checks during vehicle movement, handling zero-length junction connectors between roads, and verifying that a vehicle has reached its intended destination before

removal at a sink. Finally, excessive console logging present in the original implementation was removed to prevent extremely large outputs during long simulation runs.

2.3 Model Metrics

Model performance was evaluated using both trip-level and run-level metrics. For each vehicle that completed a trip during the simulation, the departure and arrival steps were recorded, and travel time was calculated as the difference between them, with each tick representing 1 minute. In addition, the total delay time accumulated by each vehicle at broken bridges and the origin SourceSink were stored, allowing travel times and delays to be traced back to specific parts of the network. These trip-level records were collected in the model's travel_times list and exported per scenario to a CSV file after each run.

Bridge breakdowns are determined once at model initialisation: each bridge is assigned a broken status based on its condition rating and the scenario's breakdown probabilities. When a vehicle encounters a broken bridge, it enters a waiting state for a delay sampled from a distribution dependent on bridge length, with longer bridges causing longer delays. These delays are accumulated per vehicle and recorded separately from total travel time, allowing the distinction between overall trip duration and disruption specifically caused by bridge failures. In addition, each run records the total number of broken bridges and the number in each condition class (A-D).

For each run, several aggregate metrics were computed: the number of generated trucks, the number of completed trucks, the completion rate, the 95th percentile travel time, the mean delay per truck, and the number of broken bridges. Afterward, results were aggregated per scenario and summarized using mean travel time, median travel time, standard deviation, total number of completed trips, total combined travel time, completion rate, p95 travel time, mean delay per truck, and broken bridge counts. Scenario 0, in which no bridges break down, serves as the baseline against which all other scenarios are measured.

3. Results

3.1 Experimental setup

The experimental setup consists of five scenarios labelled 0 to 4, each representing a different level of bridge breakdown probability. For each scenario, 10 independent simulation runs are performed, resulting in a total of 50 runs. For each scenario run a different random seed is used to account for stochastic variation while preserving reproducibility and comparability between scenarios. The simulation horizon is fixed at 7.200 ticks, corresponding to 5 days of simulated time, with one tick representing one minute. Across scenarios, only the bridge breakdown probabilities are varied, while all other model settings remain unchanged. For each run, the model records completed vehicle trips with their travel time and total delay time, and computes run-level metrics including the numbers of generated and completed trucks, the completion rate, the 95th percentile travel time, the mean delay per truck, and the number of broken bridges by condition, after which these results are aggregated and summarized per scenario.

3.2 Results

Before diving into the model results, let's review the data cleaning and discuss the network we created. Figure 1 shows the N1 and N2 roads with their respective side roads longer than 25 kilometers. In total, there are 24 intersection nodes representing 12 intersections with a node per road; these enable vehicles to transfer between roads during routing.

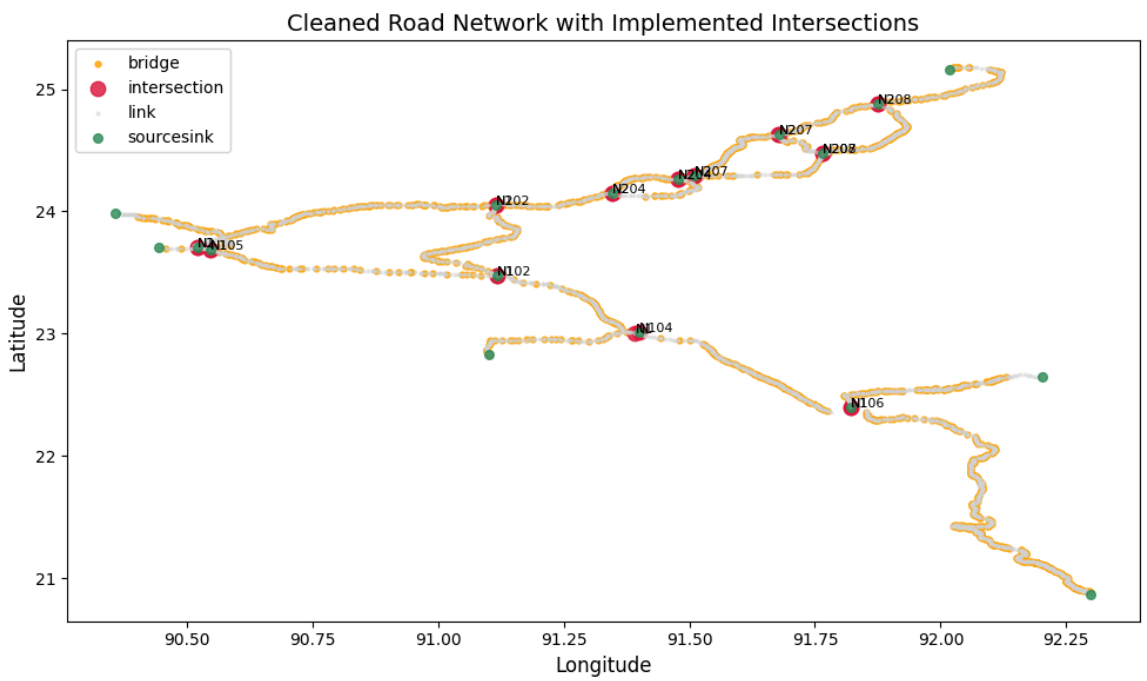


Figure 1: Plot of N1 and N2 roadnetwork with side roads.

To first get an idea of how the different scenarios affect the Bangladesh road model, data were collected on the number of broken bridges per scenario. Figure 2a shows the number of bridges broken per category, clearly showing that scenario 4 is significantly more problematic, with an average of 120 bridges breaking down. Figure 2b shows the average delay per truck for each scenario, demonstrating how the random seed affects the model: for each seed, different bridges break down, leading to different delays.

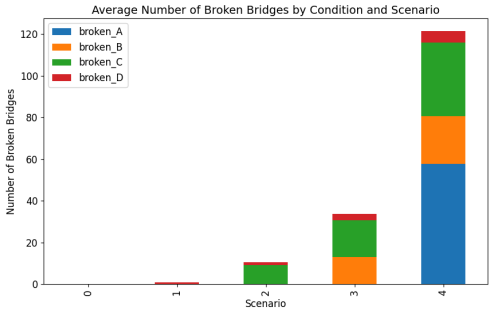


Figure 2a: Barplot of the average number of broken bridges by condition for each scenario.

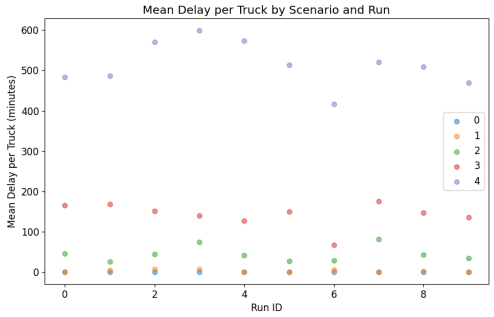


Figure 2b: Mean delay per Truck by scenario for each run.

Now that it is clear how the different scenarios affect the model, some insights can be gained into their impact on trucks driving across the network.

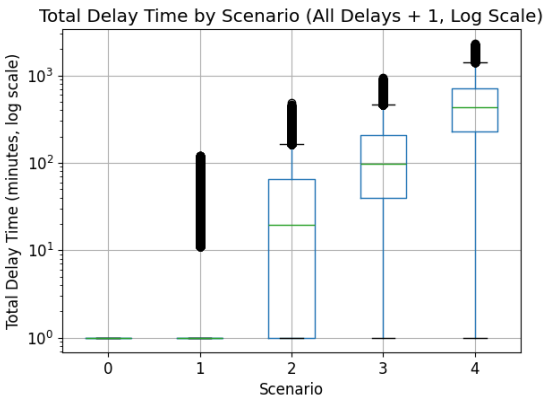


Figure 3a: Boxplot of total delay time per scenario on a +1 logarithmic scale.

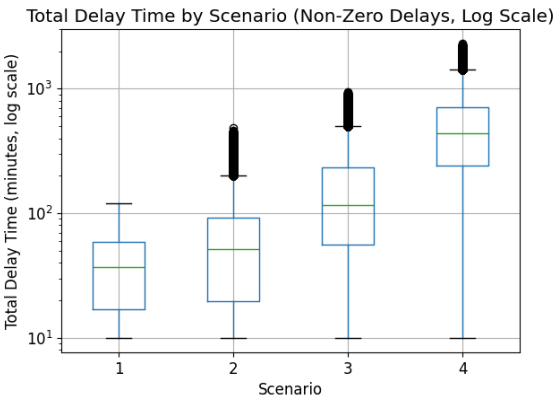


Figure 3b: Boxplot of total non-zero delay time by scenario on a logarithmic scale.

Figure 3a shows a boxplot of the delay time per truck across all 10 runs for each scenario. Figure 3b shows the boxplot for all non-zero delays, so all trucks that encountered a broken bridge and had to wait some time. Both plots use logarithmic scales because of the skewed distribution of the scenarios, with scenario 4 showing significantly more delay than the others, as shown in Figure 2b.

Combining the delay insight with the breakdown rates per bridge quality, as shown in Figure 2a, it is interesting to note that fixing the worst bridges, bridges of quality D, might not be the most effective

strategy. The results show that there are relatively few bridges of quality D breaking down in scenario 4, yet the delay is substantially larger than in the other scenarios. The primary change from scenario 3 to 4 is the probability of bridges of quality A breaking down, rising from 0 to 5%. Given the large number of additional bridge failures this introduces, it suggests that good-quality bridges may still be one of the biggest risk factors. This is because they are typically longer and represent the largest share of bridges in the N1 and N2 roadnetwork. The results show that fixing low-quality bridges might not be as efficient as ensuring high-quality bridges stay operational.

4. Reflection

In the process, several assumptions were made that affect the model's quality and the interpretation of the results. First, the way sourcesinks and intersections are added to the cleaned dataset. The raw data was cleaned using the script from the last assignment, splitting road links around bridges and adding bridges as separate network elements. Since some newly included roads ended in a bridge, we could not change the first and last points to a sourcesink; instead, we added separate sourcesink nodes at road endpoints with a negligible length of 0.001 km (1 m). This does not meaningfully affect road length and ensures no bridges are overwritten.

For intersections, we added intersection points where roads connect, using the naming convention to select side roads — including N1xx and N2xx roads longer than 25 kilometers — and checked where these interconnected with main roads. The distance-based check used for this assumes the relevant intersection point lies at a side-road endpoint rather than where it crosses the main road — a reasonable assumption given that in the Bangladesh road network, side roads mostly terminate at the main road. This works well for the selected N1 and N2 road network, but it makes assumptions about the road network and therefore may miss more complex structures if extended to smaller R and Z roads.

An additional reflection point is the creation of the intersection, where, instead of using the same road ID for intersection points on different roads, each intersection point got its own ID. This affects how the NetworkX model is initialized; this is handled by assigning 0 weights to intersections within the proximity limit, effectively putting them on top of one another. This creates the same effect, but introduces additional complexity in the model. For further data cleaning, it is better to assign the same ID to the intersection points so that NetworkX initializes the intersection automatically, rather than adjusting the weights. Since the functionality didn’t change for this assignment, it was decided not to adapt the current implementation.

Finally, we would like to reflect on how the shortest path is identified within the NetworkX model. For this assignment, the shortest path is identified based on distance, meaning each source-sink combination has a set shortest path at model initialization. This approach ensures that the delay caused by a bridge collapse does not affect the path taken. In reality, it is reasonable to assume that, given knowledge of a bridge being broken, truck drivers might consider alternative routes to avoid the bridge. This reality is not implemented in our model. Adding this dimension, where the estimated delay time based on bridges being down on the road into the shortest path can improve the model's reflection of reality and generate more valuable insights, unfortunately, it was not feasible to implement within the scope of this assignment.

5. Bonus assignment

To try to reflect whether the code-generated intersections reflect their real-life counterparts, shapefiles of the Bangladesh road network were analyzed. All roads were plotted, and a range was specified specific to the lat and lon of the intersection in the _roads3.tsv file. The roads and intersections, according to this file, were plotted over the shapefiles to see the accuracy of the roads themselves and the intersection location. These plots are included in Figure 4.

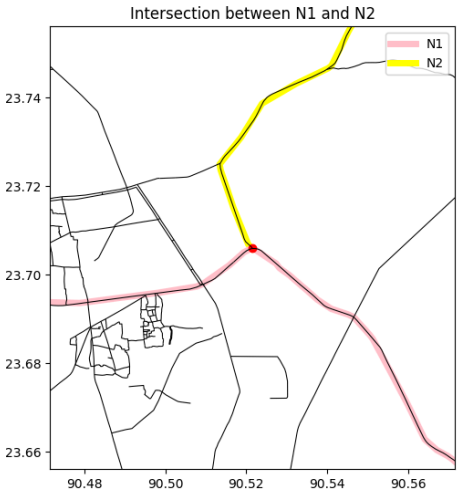


Figure 4a: intersection between N1 and N2

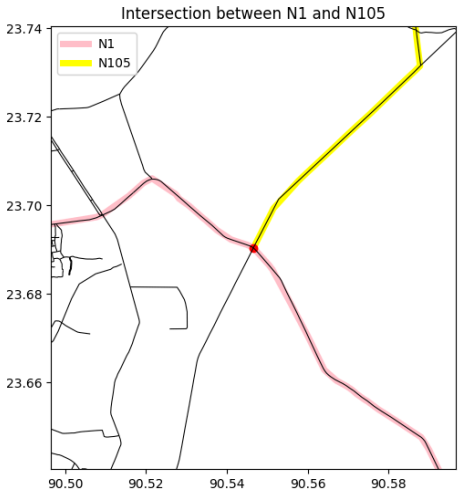


Figure 4b: intersection between N1 and N105

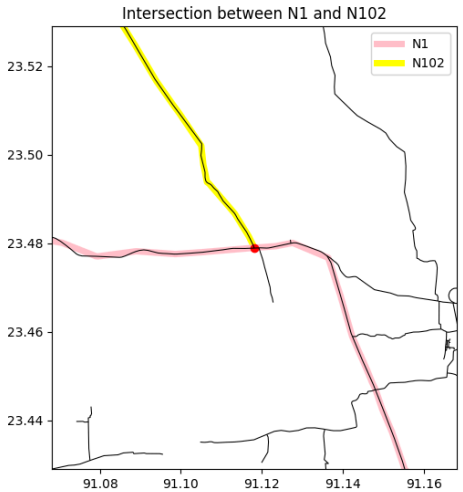


Figure 4c: intersection between N1 and N102

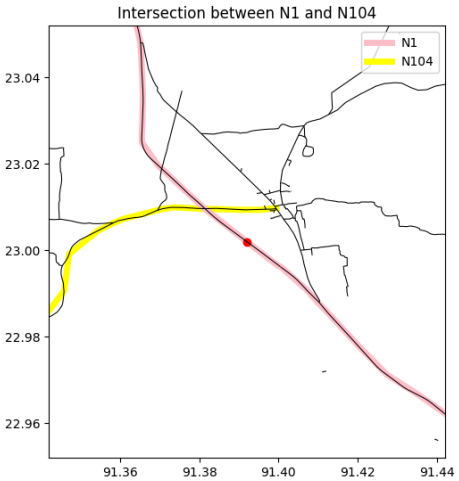


Figure 4d: intersection between N1 and N104

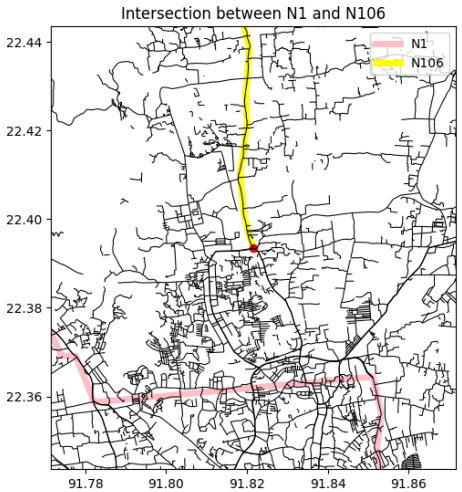


Figure 4e: intersection between N1 and N106

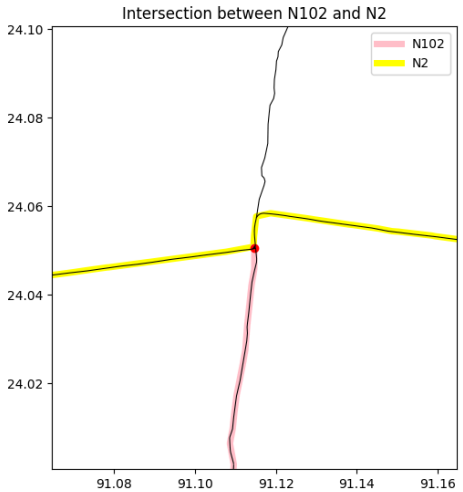


Figure 4f: intersection between N102 and N2

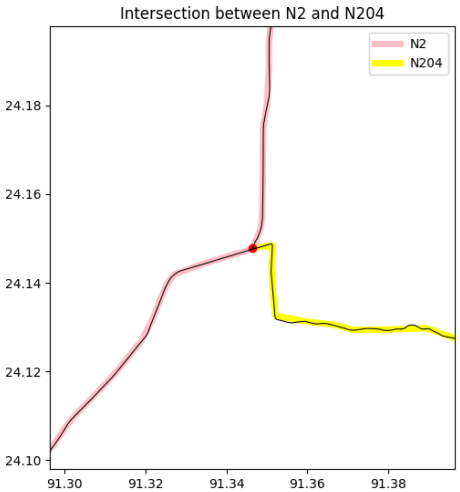


Figure 4g: intersection between N2 and N204

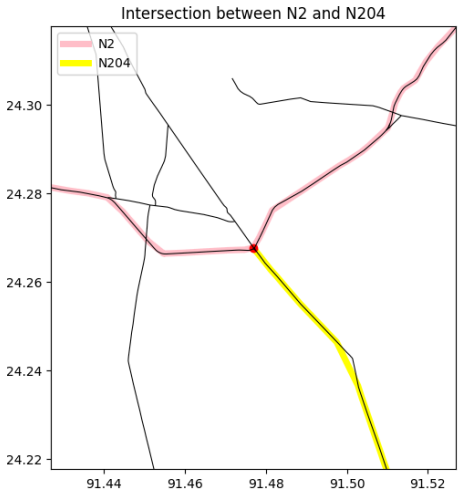


Figure 4h: intersection between N1 and N2

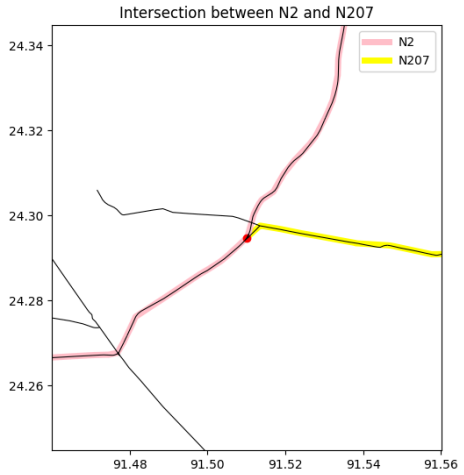


Figure 4i: intersection between N1 and N2

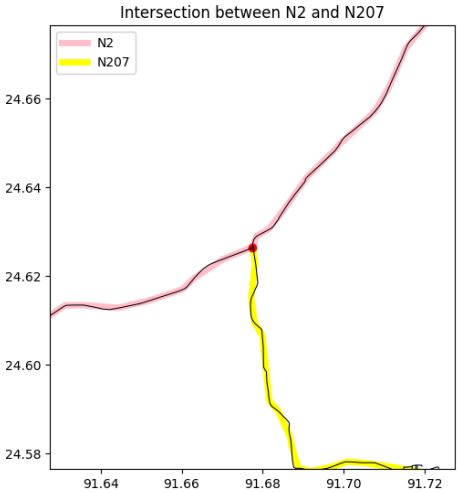


Figure 4j: intersection between N2 and N207

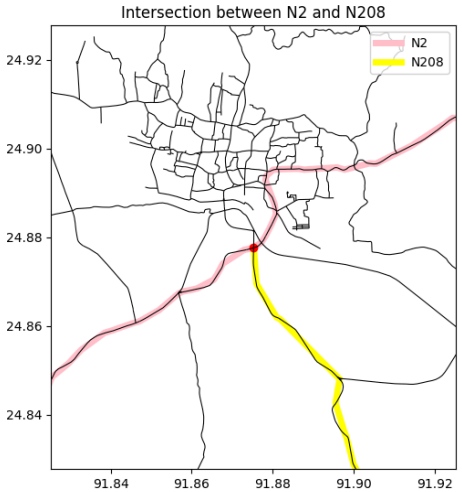


Figure 4k: intersection between N2 and N208

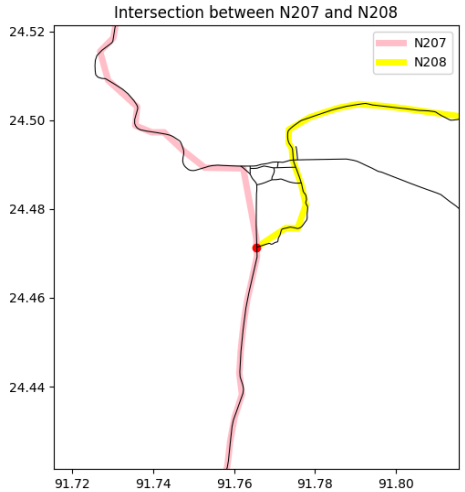


Figure 4l: intersection between N207 and N208

Figure 4: Intersections on the main roads in Bangladesh, as represented in the shapefile and calculated in the model

The plots show the twelve different intersections relevant to this assignment, specified with a red dot. The roads, according to the shapefiles, are included in black, and the roads according to the roads dataset are included in pink and yellow. Some of the plots show that the computed intersections are largely accurate. For example, Figures 4a, 4b, and 4c show a very accurate representation of the intersection. The smaller inaccuracies in the plotting of the road trajectories, visible for example in Figure 4l, will not be considered further as they don't directly influence the simulation result, and because they are only plotted for clarity on which roads are part of the intersection.

However, there are also intersections that have a wrong or ambiguous location, such as in Figures 4d and 4e. The cause of the error in the intersection calculation is very different between the two. Figure 4d shows the intersection between road N1 and road N104. For this intersection, the assumption was made that the intersection was on the endpoint of the sideroad. However, as we see here, that is not actually the case. Because the intersection is put on road N1 based on distance, it is

placed along road N1 at the point that is closest to the endpoint of road N104. That is why the intersection point is not at the same point as the shapefile indicates it should be.

To understand the inaccurate location of the intersection between N1 and N106, it helps to zoom out. Figure 5 shows that the data points of the N1 around Chittagong are very sparse. The straight line through most of the city represents the lack of data through the city, which makes it hard to accurately model the intersection. In this case, the choice was made to have the intersection be the end of the N106, and reroute the N1 to also hit this point.

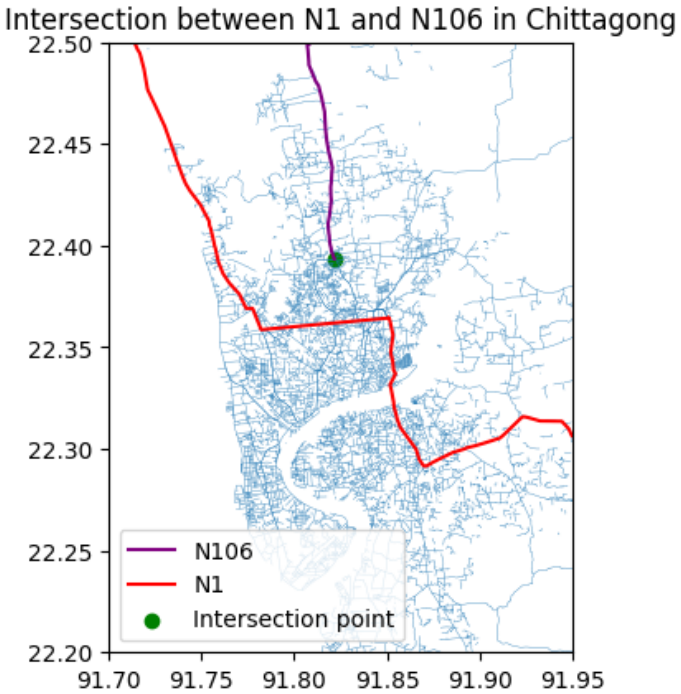


Figure 5: The intersection between N1 and N106 on a bigger scale

As shown in Figure 4, most intersection points are relatively correct, so they don't negatively affect the results, as shown in Section 4.

For the ones that are slightly wrong, you are effectively changing the length of the connected road segments. The impact would be that for calculating the shortest path, the cost of that route could go slightly up, while it shouldn't. Similarly, If moving an intersection makes Road N1 slightly longer than Road N104, your agents might suddenly switch their preferred route. In a large-scale simulation with many more agents and roads, these small errors could aggregate, potentially leading to inaccurate total delay and artificial congestion on one road or under-utilization of another, which doesn't reflect real-world behavior in Bangladesh's network.

6. Acknowledgement of AI & Group Work

6.1 AI Statement

AI tools were used as supportive instruments during this assignment. AI assisted in generating inspiration for structuring the analysis an clarifying conceptual questions. Additionally, AI was used to help identify potential coding errors, debug specific code segments, and suggest alternative implementations and/or bug fixes when troubleshooting data processing steps. All AI-generated suggestions were critically reviewed, tested, and adapted if necessary. AI was also used to help structure the code, where an implemented, working algorithm was given to the AI with the prompt to restructure it into more readable code so it could be shared with the team.
The final analysis, prioritisation of data quality issues, and implementation of cleaning algorithms reflect our own understanding and independent decision-making.
Lastly, Grammarly was used by some team members to improve the readability of their written sections.

6.2 Group Work Contribution

Group Member	**Contribution to Assignment 3**
Jonathan Vermeulen	Did some bug fixing in model implementation and data cleaning. Ran experiment and collected results. Wrote Results and Reflection.
Stijn Keukens	Made the data cleaning code, networkX routing code and DataCollection code together with Scipio. Partly wrote Introduction, Preparation, model design & Adaptation and model metrics.
Scipio Bruijn	Made the data cleaning code, networkX routing code and DataCollection code together with Stijn Partly wrote Introduction, Preparation, model design & Adaptation and model metrics.
Evi de Kok	Worked on the bonus assignment, wrote the text on that part, did a final sanity check of the document and wrote the ReadME file.
Annette Dorresteijn	Worked on the bonus assignment, wrote the text on that part and did a final sanity check of the document.